Human-Centered Neuroengineering LAB

C Programming (1)

1st week. Introduction

Department of Embedded System Engineering
Incheon National University

Instructor: Kyungho Won
kyungho.won@inu.ac.kr
Human-Centered Neuroengineering (HCN) Lab

INU 인천대학교 INCHEON NATIONAL UNIVERSITY

* These lecture slides are for class use only. Unauthorized distribution or modification is prohibited.

Instructor

Human-Centered Neuroengineering LAB

Homepage

- Kyungho Won, Ph. D.
- Human-Centered Neuroengineering Lab (HCN LAB)
  - Brain-Computer Interface, Neuroergonomics, VR
- Office: Building #9, Room 422
- Courses: 디지털회로및소자, 아날로그회로및소자, C언어프로그래밍(1), BCI 및 생체신호 개론 (대학원)
- Contact: kyungho.won@inu.ac.kr

Educational & professional backgrounds

- 2012 Mar – 2016 Feb
- Embedded System Engineering

- 2014 Jan – 2014 Oct
- Exchange program

- 2016 Mar – 2022 Aug
- Electrical Engineering and Computer Science
- ***End-to-End Brain-Computer Interfaces (BCIs)***

- 2021 Oct – 2022 Apr
- Visiting scholar to Neuroergonomics lab

- 2022 Sep – 2023 Feb
- Postdoc

- 2023 Mar – 2025 Jan
- Postdoc
- ***BCI for immersive VR experience***

- 2025 Mar – present
- Assistant Professor @ Embedded System Engineering Dept.
- ***Human-Centered Neuroengineering (HCN) Lab***

인간중심 신경공학 연구실
Human-Centered Neuroengineering (HCN) Lab

원 경 호 교수

- **Education**
  - 2022: Ph.D. in Electrical Engineering and Computer Science (EECS), GIST
  - 2016: B.S. in Embedded System Engineering, Incheon National University
- **Professional Experience**
  - Postdoc. @ Hybrid Team, Inria Rennes, France
  - Postdoc. @ BioComputing Lab, GIST

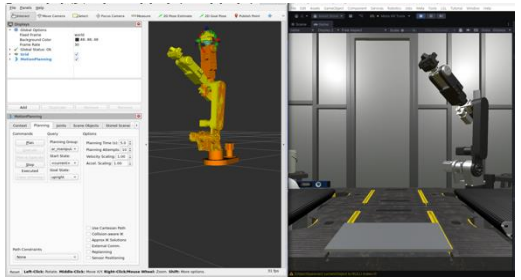
Homepage

연구 분야

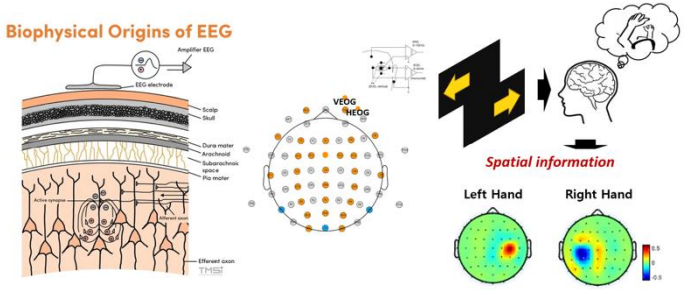
뇌파 (뇌전도, EEG) 측정

가상 현실 (VR) 기반 설계

인간-로봇 상호 작용 (HRI)



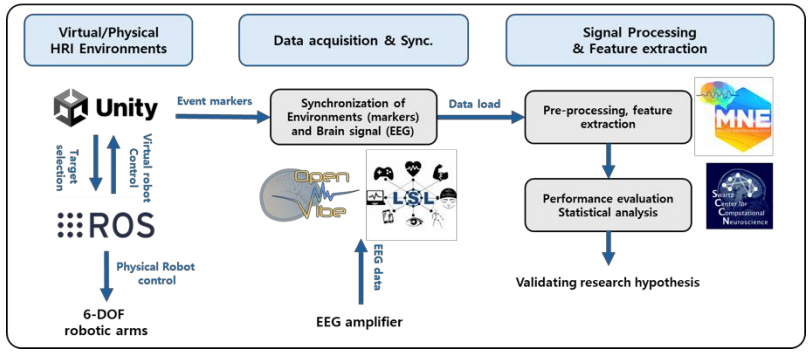
디지털 트윈: 가상-실제 로봇 동기화



AI 기반 뇌파 분류
(예: Brain-Computer Interface, BCI)

연구실 비전 / 학부·대학원 과정 지도

Human-Robot Interaction (HRI) Framework with Brain



2025 KEEG 학술대회 참가

- 다양한 실험 장비 (뇌전도 측정 장치, 6축 로봇팔, VR/AR 헤드셋) 활용 및 실제 생체 신호 데이터 획득 과정에 참여
- Multimodal 데이터를 동기화해 다양한 관점에서 탐구할 수 있는 능력 함양
- 가상현실 (VR) – 로봇 (ROS) 간 동기화를 통한 디지털 트윈 구현 기술 학습
- 패턴인식 (AI) 기술을 활용한 생체 (뇌전도, 심전도, 근전도) 분류 기술
- 국내외 학술대회, 워크숍 참가를 통한 타 연구자와의 연구 교류, 연구 동향 파악

연구 실적

- 국내외 학술대회 및 학술지 논문 다수 게재
- Graz BCI, International BCI meeting 등 연구동향 파악을 위한 학술대회 정기 참가

Contact

- 원경호 교수 (kyungho.won@inu.ac.kr)
- 교수 연구실: 9호관 (공동실험실습관) 422호
- 연구실/실험실: 9호관 (공동실험실습관) 405B호

인천대학교 INCHEON NATIONAL UNIVERSITY

General Course Policy

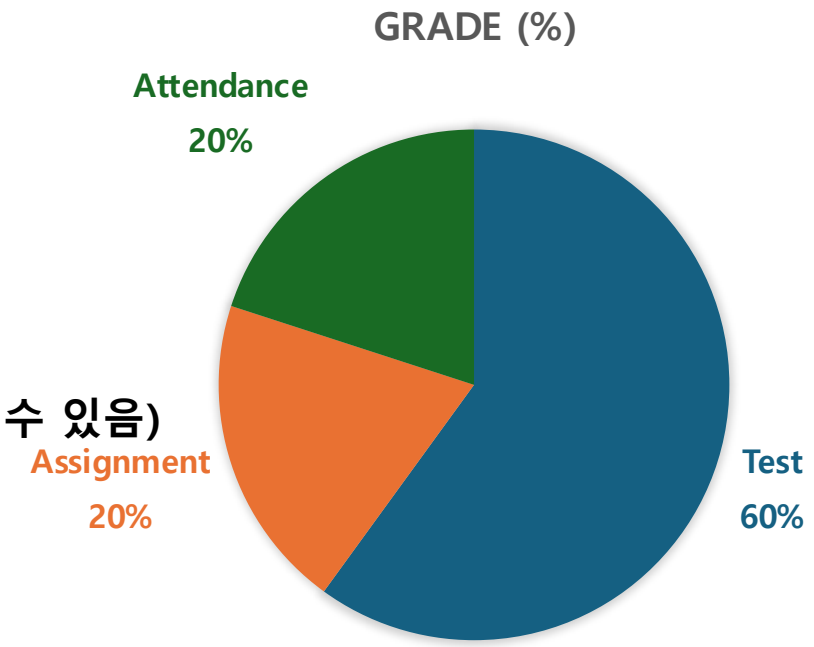
- Class hours (306)
  - Tue. 11h-13h / Thr. 16h-18h

- TA
  - 황주옥 (ghkdwndhr227@inu.ac.kr), 010-6403-1470
  - 연락 시 00 과목을 수강중인 000 입니다 밝힌 후 질의

- Office hours (instructor's office: Building #9, Room 422)
  - Appointment is necessary in advance
  - E-mail: kyungho.won@inu.ac.kr
  - 연락 시 00 과목을 수강중인 000 입니다 밝힌 후 질의

General Course Policy

- Grade (상대평가)
  - Exam A/Exam B/Exam C (Test or Projects): 60 %
  - Assignments: 20 %
  - Attendance: 20 %
    - 1시간 결석 시 1/3점 감점, 3시간 결석 시 1점 감점
    - 실제 수업시간 수의 1/3이상 결석: 학점인정 불가



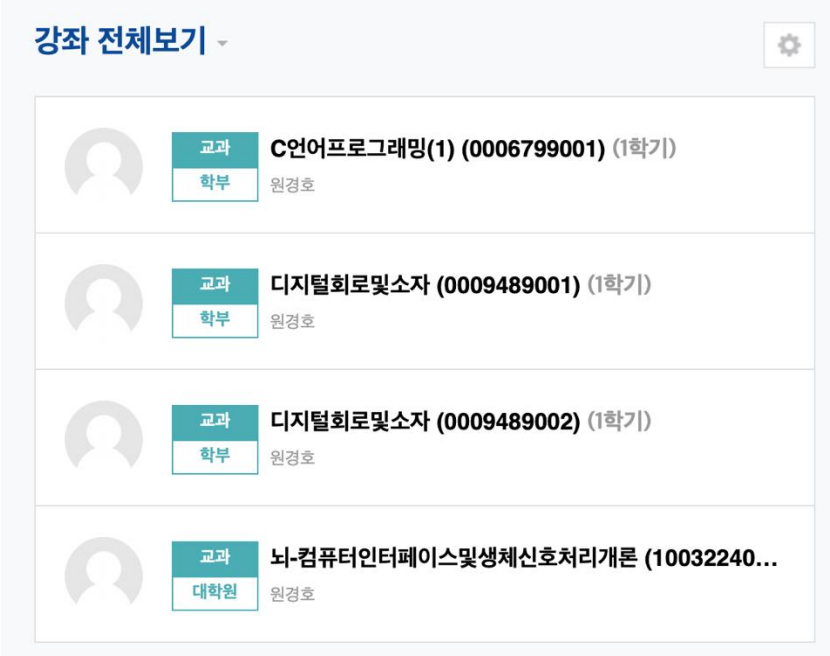
- 상대평가 등급 비율 (정정기간 후 최종 수강인원에 따라 일부 조정될 수 있음)

A+/A0등급	B+/B0등급	C+ 이하	F
상위 35 %	상위 35 % ~ 75%	75 % - 100 %	출결/과제 성실성

** A+/A0, B+/B0는 절반씩 배분

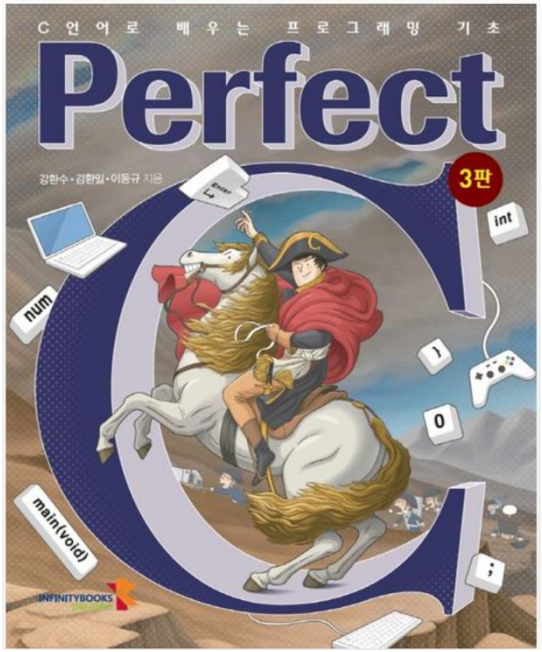
General Course Policy

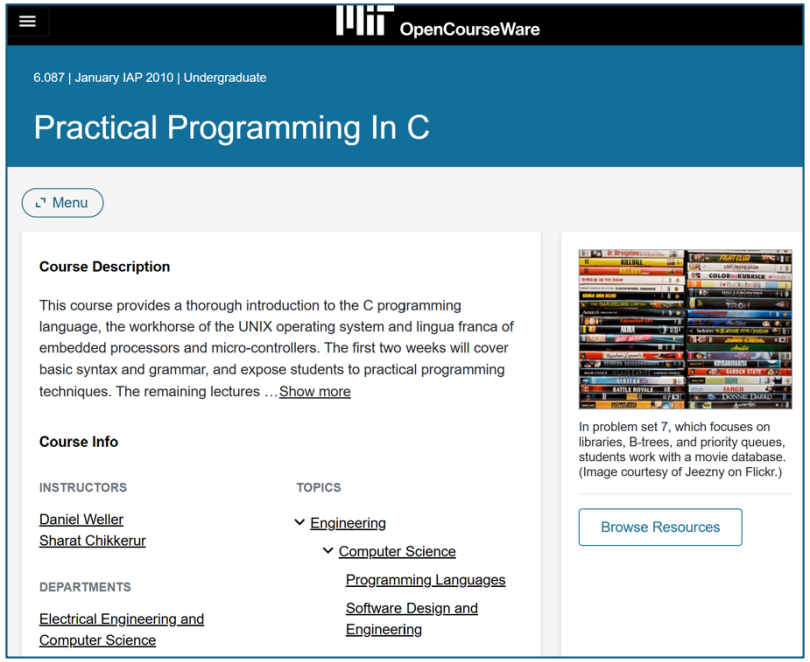
- e-learning site; 인천대 포탈 로그인 > 이러닝 선택 > 수강중인 강좌 선택
- App: 인천대학교 스마트캠퍼스, 코스모스 앱 다운로드 및 로그인
- 강의자료 및 과제, 기타 공지사항 확인



Class Materials

- Lecture slides
  - Slides (PDF) will be distributed 1-hour earlier
- Perfect C 3판
  - 강환수, 강환일 이동규 저
  - 인피니티북스
- Practical Programming in C
  - Daniel Weller, Sharat Chikkerur
  - MIT OpenCouseWare





Course Syllabus (tentative)

1st week 09.01 ~ 09.05	- Introduction, Hello World, Compiling & Debugging	
2nd week 09.08 ~ 09.12	- Variables and Data Types	
3rd week 09.15 ~ 09.19	- Operators, Expressions, Preprocessor & I/O	
4th week 09.22 ~ 09.26	- Control Flow – Conditions (if, switch)	
5th week 09.29 ~ 10.03	- Control Flow – Loops (for, while, do-while)	
6th week 10.06 ~ 10.10	- Arrays	
7th week 10.13 ~ 10.17	- Functions	
8th week 10.20 ~ 10.24	Midterm	

Course Syllabus (tentative)

9th week 10.27 ~ 10.31	- Variable Scope, Static & Global Variables	
10th week 11.03 ~ 11.07	- Pointers Basics, Memory Addressing	
11th week 11.10 ~ 11.14	- Characters and Strings	
12th week 11.17 ~ 11.21	- Structs and Unions	
13th week 11.24 ~ 11.28	- Functions & Pointer Applications	
14th week 12.01 ~ 12.05	- File Processing + C Standard Library	
15th week 12.08 ~ 12.12	Final	
16th week 12.15 ~ 12.19	- 보강 주	

Course Structure

Weekly format:

- 2 hours: Lecture (theory + examples)
- 2 hours: Lab (hands-on practice)

This week's lab:

- Install IDE (Visual Studio / VS Code)
- Write, build, and run Hello World
- Complete practice problems

Today's Agenda

- What is C and why learn it?
- C language features and history
- Development environment setup (IDE)
- Your first program: Hello World
- Understanding errors and debugging
- C for embedded systems
- Practice problems

What is C?

What is a Program?

그림 1-1 일상생활에서 활용하는 스마트폰 프로그램 "앱"과 컴퓨터의 여러 프로그램들

Programs are everywhere — apps, games, operating systems

Programming languages

- A language used to develop programs
  - Communication between humans and computers
  - Designed to describe instructions given to a computer
- Various programing languages
  - FORTRAN, ALGOL, BASIC, COBOL, PASCAL, C, C++, Visual Basic
  - Java, Objective-C, JSP, Javascript
  - Python, C#, Go, Swift, Kotlin

그림 1-5 프로그램을 개발하는 프로그래밍 언어

What is C?

- C is a general-purpose programming language
- Created by Dennis Ritchie (1972)
  - At AT&T Bell Labs
  - Originally for UNIX operating system
  - The language was designed to achieve performance comparable to assembly language while being easier to use and portable across different CPU architectures.
- Still widely used 50+ years later!

What is C Used For?

- Operating systems: Linux kernel, Windows core
- Embedded systems: IoT, automotive, medical devices
- System software: Compilers, databases
- Games: Performance-critical game engines
- Microcontrollers: Arduino, STM32, AVR, ...

Features of C

- Procedural language
  - Function-centered procedural language
  - Composed of multiple functions (procedures)
  - Structured programming: decompose problems into functions and data
- Concise & Efficient language
  - Rich system libraries readily available
  - Diverse operations; bitwise, increment/decrement, compound assignments, etc.
- Highly portable language
  - Compilers available for various CPUs and platforms
  - Write once, compile anywhere with minimal changes

History of C

- 1972: C invented by Dennis Ritchie
- 1978: 'The C Programming Language' (K&R book)
- 1989: ANSI C / C89 — first standard
- 1999: C99 — added new features
- 2011: C11 — latest widely used standard

Development Environment

What You Need

- 1. Code Editor / IDE
  - Writes and organizes your source code
- 2. Compiler
  - Translates C code → machine code
- 3. Debugger
  - Helps find and fix errors

- We use an IDE that bundles all three!

What Does a Compiler Do?

Machine code

- The only language that a computer can directly understand
- Represented by 1s and 0s, indicating the presence or absence of electrical signals

Compiler

- Translate programming languages into machine language
- The compiler converts high-level programming code into machine-readable code

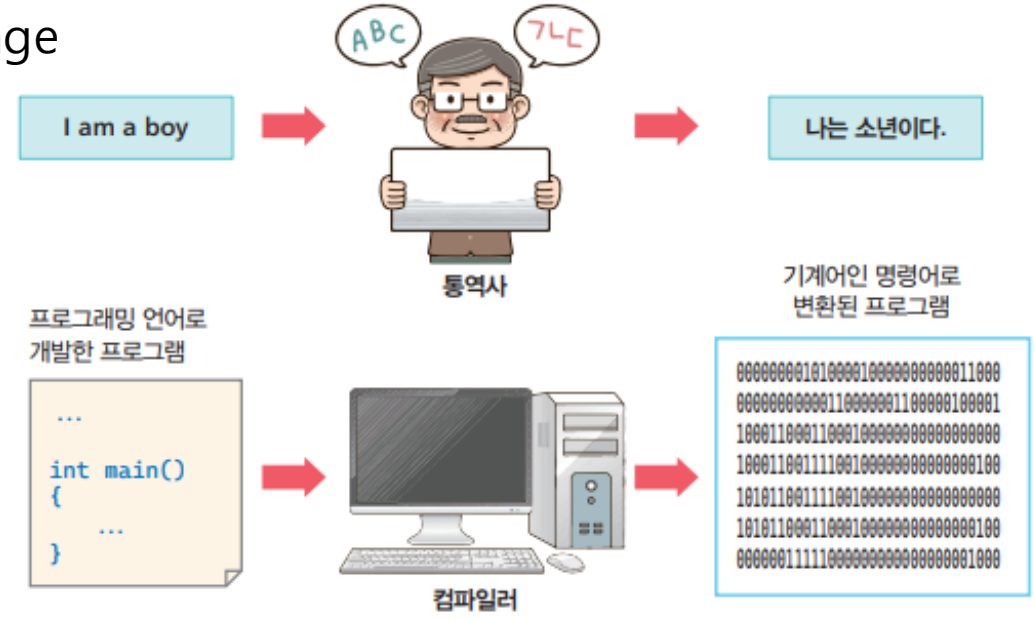


그림 1-10 통역사와 컴파일러

Compiler = translator: converts C (human-readable) into machine code

Low-level and high-level languages

Low-Level Language

- Closely related to hardware
- Includes machine language and assembly language
- CPU-dependent and not interchangeable across architectures
- Provides direct control over hardware resources

High-Level Language

- Designed to be easily understood by humans
- Abstracts away hardware details > Hardware-independent
- Requires a compiler or interpreter to translate into machine language

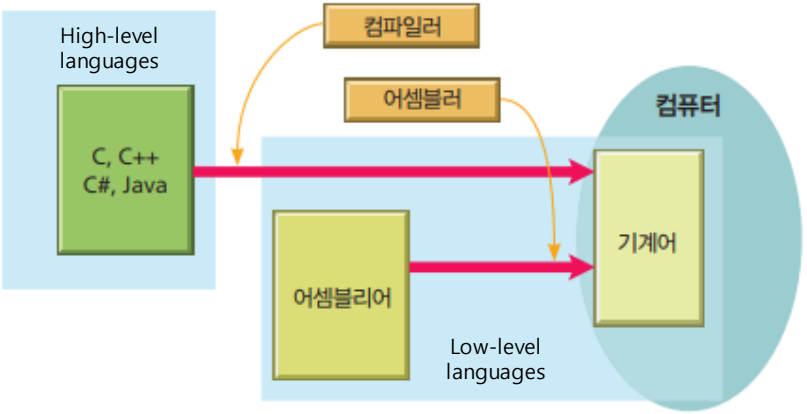


그림 1-15 컴파일러와 어셈블러

What Does a Compiler Do?

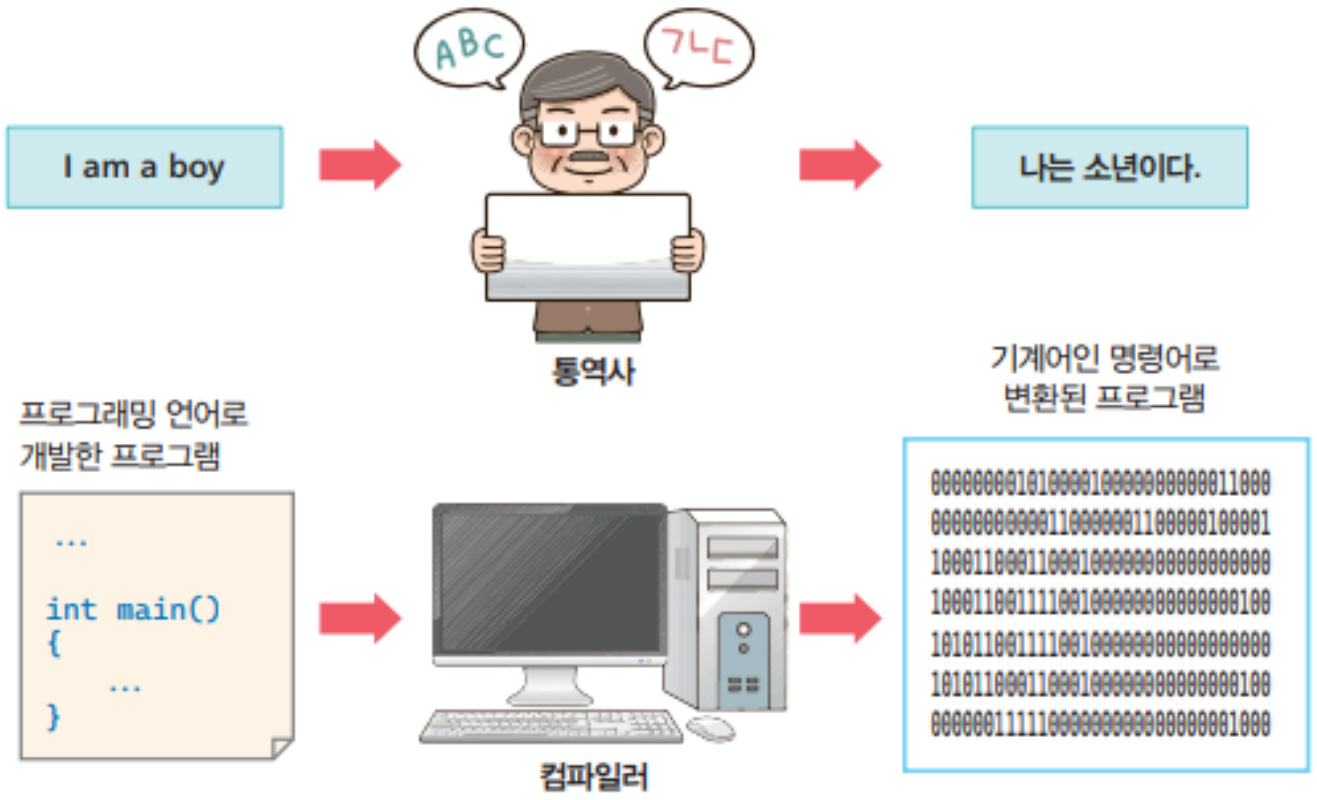
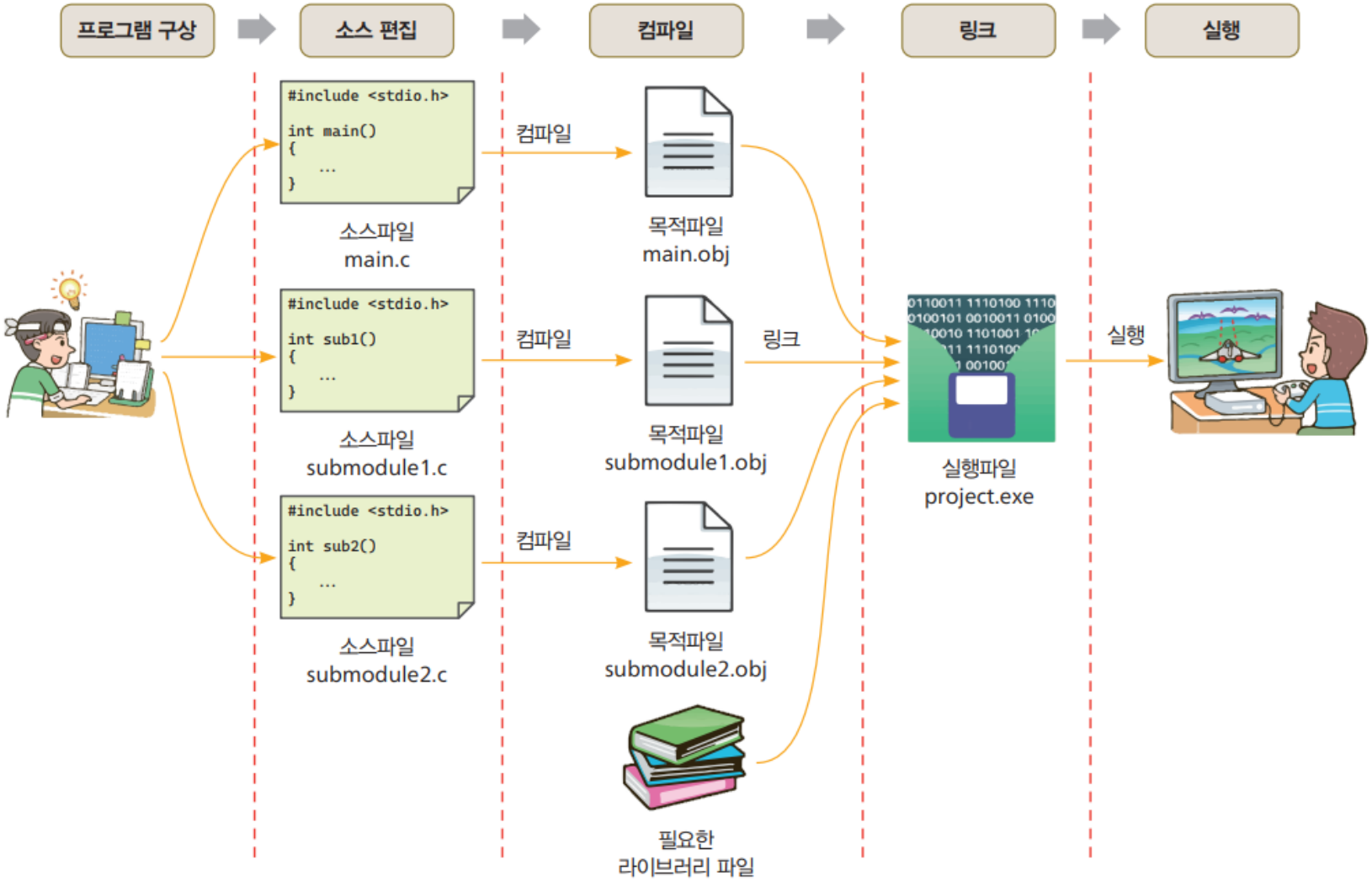


그림 1-10 통역사와 컴파일러

Compiler = translator: converts C (human-readable) into machine code

From Source Code to Execution



Source editing → Compile → Link → Execute

Compilation Steps

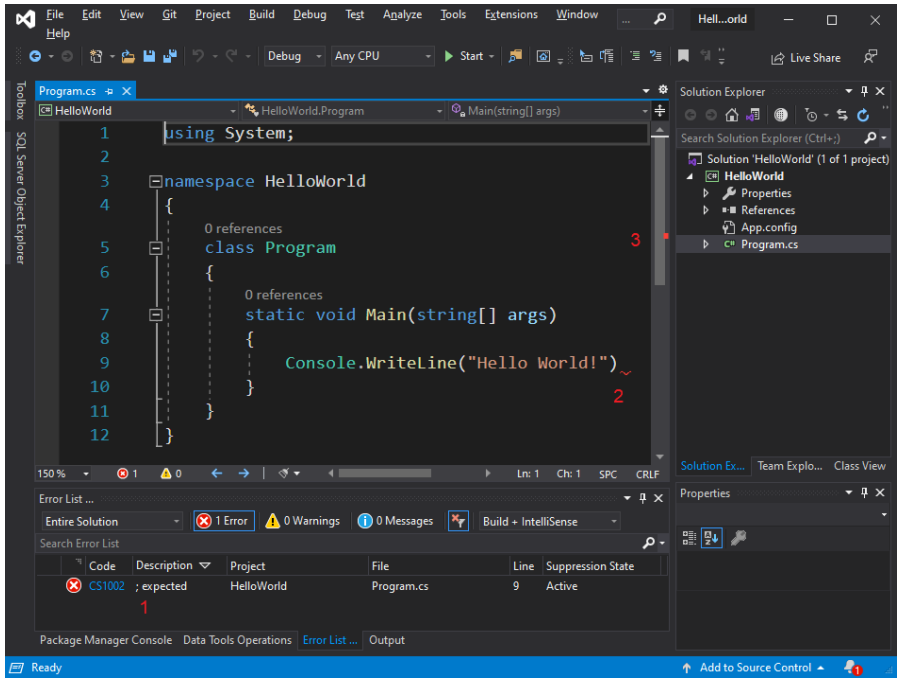
- 1. Preprocessing — handle #include, #define
- 2. Compilation — C source → assembly code
- 3. Assembly — assembly → object file (.obj)
- 4. Linking — combine objects → executable (.exe)

- Your IDE does all of this with one button click!

What is a Library?

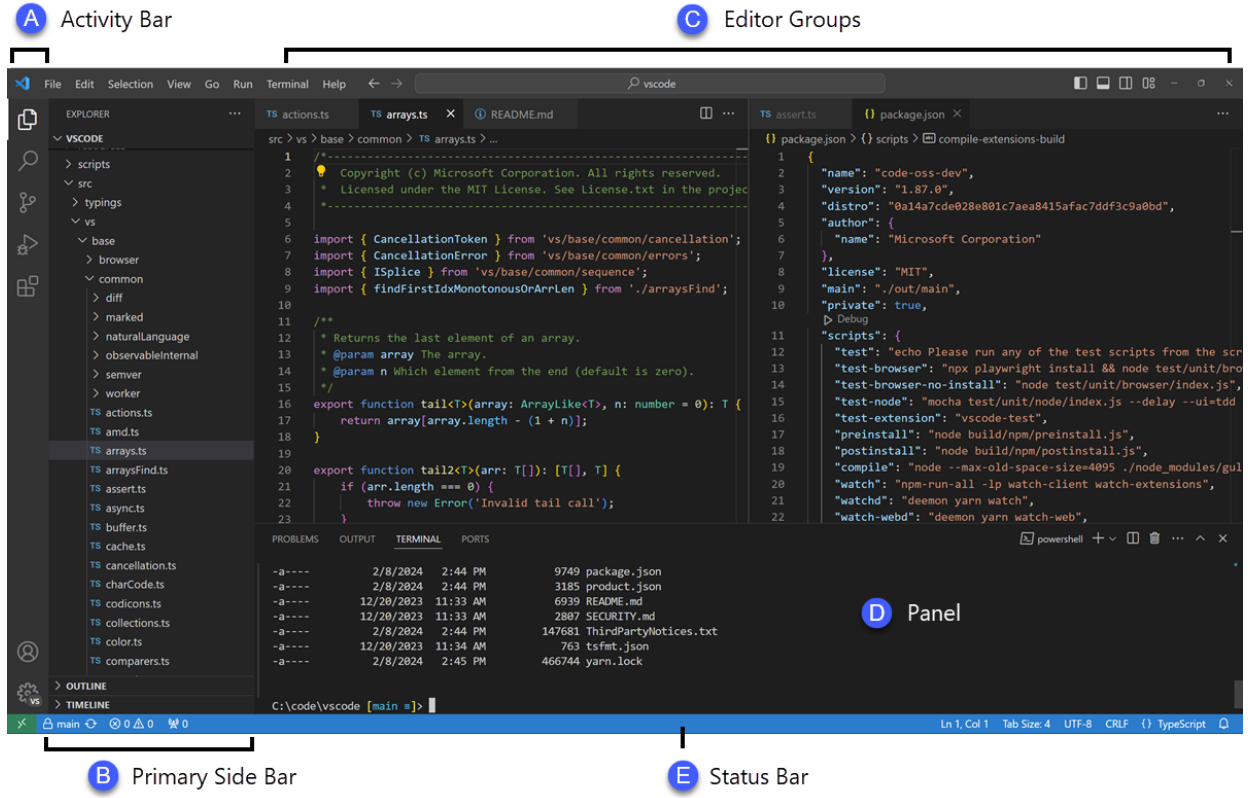
- A collection of pre-built, reusable code modules
  - Instead of writing everything from scratch, use ready-made functions
- Analogy:
  - Like using a library's books when writing a report
- In C programming:
  - printf(), scanf(), sqrt() — all come from libraries
  - The linker connects your code with these library functions
  - Standard libraries are included with every C compiler

IDE & Code editor (used in this course) (Independent development environment)



Visual studio (IDE)

full IDE with built-in compiler, debugger, and build system. Click and run.



Visual studio code (code editor)

lightweight editor. Need to install compiler (gcc/clang) and configure everything yourself.

Windows: Visual Studio

- During installation, select:
- • 'Desktop development with C++'
- • (includes C compiler + debugger)

- Recommended for most Windows users
- Large but full-featured professional IDE

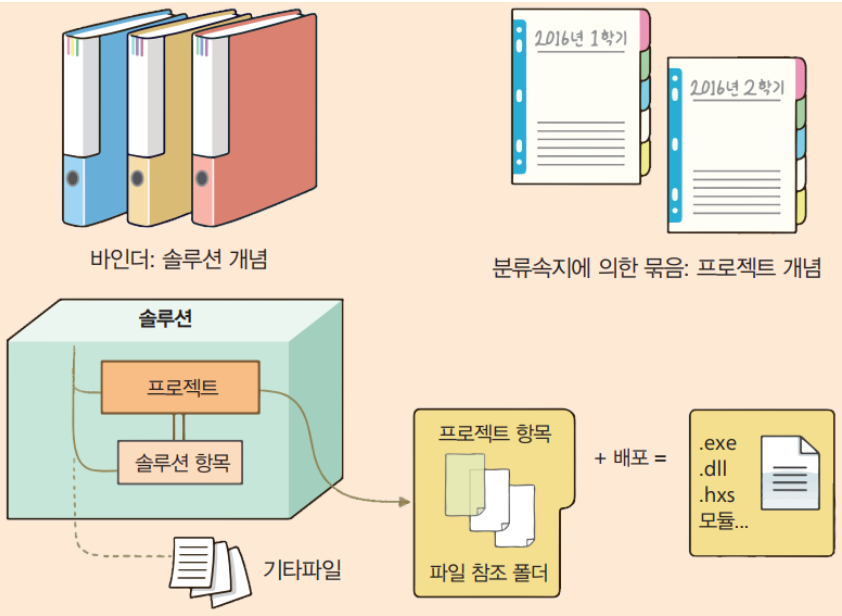
Visual Studio: First Project

1. File → New → Project
2. Select: 'Empty Project' (C++)
3. Project name: HelloWorld
4. Right-click Source Files → Add → New Item
5. Choose: C++ File (.cpp) name it: hello.c
6. Write your code and press Ctrl+F5 to run

- Tip: Save your file with .c extension!

Solution & Project in Visual Studio

- Visual Studio organizes code using Solutions and Projects
- Solution = a binder (container)
  - Holds one or more projects, like a folder for a course
- Project = a tab divider inside the binder
  - Contains source files for one program
  - Each project produces one executable (.exe)
- Example: Solution 'ch01' contains
  - Project 'P01-HelloWorld' → hello.c → P01-HelloWorld.exe
  - Project 'P02-Practice' → practice.c → P02-Practice.exe



Mac: Visual Studio Code

- Required extensions (install from Extensions tab):
- • C/C++ by Microsoft

  → Syntax highlighting, IntelliSense, debugging
- • Code Runner by Jun Han

  → Run C files with one click (▶ button)

VS Code: Build and Run

1. Open VS Code
2. File → Open Folder → choose your project folder
3. Create a new file: hello.c
4. Write your C code
5. Press the ▶ (Run) button or Ctrl+Alt+N

- Output appears in the terminal panel at the bottom
- Tip: Make sure to save (Cmd+S) before running!

The Write → Build → Run Cycle

- Write: Type C code in the editor

  ↓

- Build: IDE compiles and links (Ctrl+F5 / ▶)

  ↓

- Run: See the output

  ↓

- Error?: Read the error message, fix, and rebuild

- You will do this thousands of times — get comfortable!